Instituto Tecnológico de Estudios Superiores de Monterrey

Maestría en Inteligencia Artificial Aplicada

Título del trabajo

“Proyecto MiBici: MLflow Project, Tracking Avanzado y Model Registry”

Nombre de los Integrantes

Jesus Roberto Tafolla Aranda - A01703178

Gemma Alejandra Martínez Santos - A01652255

Sebastian Yair Mucito Nájera - A01797619

Gerardo Carreño García - A01797668

Materia

Operaciones de aprendizaje automático - TC5061.10

Fecha de entrega: 25 de Octubre del 2026

Repositorio: https://github.com/laloz60/TC5061-Proyecto-Equipo01

Contenido

1. Introducción 2
1.1 Punto de partida 2
2. Parte A. Proyecto reproducible 2
2.1 Servidor de tracking con base de datos 2
2.2 Archivo MLproject y entorno declarado 3
2.3 Reproducción con mlflow run 3
3. Parte B. Experimentación y artefactos 4
3.1 Diseño de la experimentación 4
3.2 Tabla comparativa por pregunta 5
3.3 Análisis de la comparación 7
3.4 Registro de artefactos 7
4. Justificación de la selección del mejor modelo 9
4.1 P1 — Demanda: Random Forest (n_estimators=200) 9
4.2 P2 — Riesgo: Regresión logística (C=1.0, class_weight=balanced) 9
4.3 P3 — Grupos: K-Means++ (K=3, n_init=10) 10
5. Parte C. Model Registry y ciclo de vida 10
5.1 Modelos registrados 10
5.2 Esquema de ciclo de vida: aliases 10
6. Evidencia en la interfaz de MLflow 11
6.1 Comparación de corridas con nested runs 11
6.2 Detalle de artefactos de una corrida candidata 13
6.3 Modelos registrados en producción con su linaje 15
7. Flujo de trabajo con Git y GitHub 16
8. Conclusiones 17
9. Declaración de uso de inteligencia artificial 17
Referencias 18

1. Introducción

En esta segunda fase del proyecto integrador MiBici se llevó el uso de MLflow a un nivel profesional sobre el mismo repositorio (`TC5061-Proyecto-Equipo01`), el mismo experimento (`mibici-equipo01`), los mismos datos versionados y las mismas tres preguntas de la Propuesta de proyecto:

- **P1 — Demanda (regresión):** ¿cuántos viajes se iniciarán en una estación en un periodo futuro?
- **P2 — Riesgo (clasificación):** ¿una estación estará en riesgo de quedarse sin bicicletas o anclajes?
- **P3 — Grupos (clustering):** ¿qué grupos de estaciones comparten patrones de uso y dónde se ubican?

Los objetivos de la entrega fueron cuatro: (1) empaquetar el código como un **MLflow Project** ejecutable con un solo comando; (2) comparar sistemáticamente múltiples experimentos por pregunta contra la línea base registrada en la entrega anterior, incluyendo búsquedas organizadas con *nested runs*; (3) registrar en cada corrida candidata los artefactos que respaldan su resultado, incluida la versión de los datos; y (4) llevar el mejor modelo de cada pregunta al **Model Registry** y gestionar su ciclo de vida hasta producción. Este enfoque atiende directamente la deuda técnica que Sculley et al. (2015) identifican en los sistemas de ML —dependencias de datos no declaradas, configuración dispersa y experimentos difíciles de reproducir— y se alinea con las prácticas de reproducibilidad y gobernanza descritas por Treveil et al. (2020).

1.1 Punto de partida

La entrega parte de lo construido en la Propuesta de proyecto, que sigue vigente:

- Ingesta reproducible en `mibici/dataset.py`, que descarga 41 CSV mensuales (2023-05 a 2026-09, 15,376,127 viajes y 484 estaciones) y genera `data/manifest.json` con el hash SHA-256 de cada archivo.
- *Feature engineering* sin fuga temporal en `mibici/features.py` y configuración centralizada (rutas, semilla `SEED=42`, cortes de fecha) en `mibici/config.py`.
- Un *baseline* por pregunta registrado en `mibici-equipo01`, con la partición temporal train 2023-05..2024-12, validación 2025-01..2025-06 y prueba 2025-07..2026-09.

2. Parte A. Proyecto reproducible

2.1 Servidor de tracking con base de datos

El Model Registry requiere un almacenamiento respaldado por base de datos, por lo que se levantó un servidor local sobre SQLite y todas las corridas apuntan a él mediante `MLFLOW_TRACKING_URI`. Ambos comandos están documentados en el README, con variantes para bash, PowerShell y CMD:

```
# Terminal 1: servidor (UI en http://127.0.0.1:5000)
mlflow server --backend-store-uri sqlite:///mlflow.db --port 5000

# Terminal 2: apuntar las corridas al servidor
export MLFLOW_TRACKING_URI="http://127.0.0.1:5000"     # bash / macOS / Linux
$env:MLFLOW_TRACKING_URI = "http://127.0.0.1:5000"     # PowerShell
set MLFLOW_TRACKING_URI=http://127.0.0.1:5000          # CMD (sin espacios)
```

El archivo `.gitignore` excluye `mlflow.db`, `*.db`, `mlruns/`, `mlartifacts/`, el contenido de `data/` (salvo `data/manifest.json`), los modelos serializados (`*.pkl`, `*.joblib`, `*.onnx`, `*.bin`) y los archivos `.env`. Así, el repositorio no contiene datos de MiBici, bases de MLflow, modelos ni credenciales.

2.2 Archivo MLproject y entorno declarado

En la raíz del repositorio se definió el archivo `MLproject` (nombre `mibici-equipo01`) con un entorno `python_env.yaml` que fija Python 3.12.9 y las mismas versiones del archivo de dependencias (`mlflow==3.17.0`, `scikit-learn==1.9.1`, `pandas==3.0.6`, `numpy==2.5.4`, entre otras). El `MLproject` es configuración pura: cada punto de entrada delega en `mibici/mlflow_entry.py`, y los valores por defecto de rutas, fechas y semilla se leen de `mibici/config.py` sin repetirse.

Tabla 1. *Puntos de entrada declarados en el archivo MLproject.*

Entry point	Parámetros	Qué ejecuta
`ingest`	`force`, `skip_download`, `limit`	Descarga y unificación de los CSV de MiBici; escribe el Parquet y `data/manifest.json` (`mibici/dataset.py`).
`features`	—	Genera `features_estacion_hora.parquet` con lags y promedios que solo usan horas anteriores.
`train`	`question` (p1 | p2 | p3), `model`, `n_estimators`, `rf_max_samples`, `n_clusters`, `n_init`	Entrena la pregunta indicada con el modelo e hiperparámetros recibidos y registra métricas, etiquetas y artefactos en MLflow.
`search`	`question` (p1 | p2 | p3), `grid`	Búsqueda de hiperparámetros con *nested runs*: una corrida padre y una hija por valor de la malla (P1: `n_estimators`; P2: `C`; P3: K). `grid=auto` usa la malla definida en `train.py`.
`register`	—	Selecciona el mejor run de cada pregunta, lo registra en el Model Registry con su linaje y le asigna el alias `@champion` (`mibici/modeling/register.py`).

2.3 Reproducción con mlflow run

Todas las corridas de la entrega se lanzan desde el MLflow Project, no desde un notebook. Desde la raíz de un clon limpio, con el servidor activo y `MLFLOW_TRACKING_URI` definida, la secuencia documentada en el README es:

```
# 1. Datos y features
mlflow run . --experiment-name mibici-equipo01 --env-manager uv -e ingest  # Win: -P limit=0
mlflow run . --experiment-name mibici-equipo01 --env-manager uv -e features

# 2. Baselines
mlflow run . --experiment-name mibici-equipo01 --env-manager uv -e train -P question=p1 -P model=ridge
mlflow run . --experiment-name mibici-equipo01 --env-manager uv -e train -P question=p2 -P model=dummy
mlflow run . --experiment-name mibici-equipo01 --env-manager uv -e train -P question=p3 -P model=random

# 3. Búsquedas con nested runs (de aquí salen los modelos elegidos)
mlflow run . --experiment-name mibici-equipo01 --env-manager uv -e search -P question=p1
mlflow run . --experiment-name mibici-equipo01 --env-manager uv -e search -P question=p2
```

```
mlflow run . --experiment-name mibici-equipo01 --env-manager uv -e search -P question=p3

# 4. Model Registry
mlflow run . --experiment-name mibici-equipo01 --env-manager uv -e register
```

`--env-manager uv` crea el entorno aislado del `python_env.yaml`, y `--experiment-name` garantiza que ninguna corrida caiga en el experimento `Default`. El punto de entrada `search` hace que también las búsquedas anidadas, de las que salen los tres modelos en producción, se lancen desde el proyecto; `-P grid=...` permite cambiar la malla (por ejemplo, `-P question=p3 -P grid=3,4`). La secuencia completa se validó el 22 de octubre de 2026 en un clon nuevo (Windows 10, CMD, Python 3.12): los once pasos terminaron con éxito, incluida la carga de un modelo registrado y la suite de `pytest` (29 pruebas). La bitácora con cada comando y su salida está en `reports/validacion_clon_limpio_issue62.md`. Durante esa validación se documentaron dos ajustes para Windows (activar el Modo de programador para los *symlinks* de `uv` y pasar `-P limit=0` a la ingesta) y se corrigió un aviso engañoso de re-ingesta (issue #86).

3. Parte B. Experimentación y artefactos

3.1 Diseño de la experimentación

Para que la comparación contra el *baseline* fuera válida se mantuvieron la misma partición temporal y las mismas métricas de la entrega anterior: MAE (además de RMSE y R^2) para P1, ROC-AUC y F1-macro para P2, y coeficiente de silueta para P3. La selección de hiperparámetros se hizo en validación y cada modelo elegido se evaluó una sola vez en prueba; en P2 se corrigió en esta entrega una fuga que usaba prueba para elegir `C` (PR #79).

Cada pregunta cuenta con una búsqueda de hiperparámetros organizada como *nested runs*: una corrida padre por búsqueda y una corrida hija por combinación.

Tabla 2. *Búsquedas de hiperparámetros con nested runs.*

Pregunta	Corrida padre	Corridas hijas (candidatas)	Espacio de búsqueda
P1	`p1-parent-search-rf`	`p1-candidate-rf-n50`, `-n100`, `-n200`	`n_estimators` ∈ {50, 100, 200}
P2	`P2-parent-search-logreg`	`p2-child-logreg-C0.1`, `-C1.0`, `-C10.0`	`C` ∈ {0.1, 1.0, 10.0}, `class_weight=balanced`
P3	`p3-parent-search-kmeans`	`p3-candidate-k3` … `p3-candidate-k6`	K ∈ {3, 4, 5, 6}, `n_init=10`

Además de las etiquetas obligatorias de equipo, pregunta y *commit* (`equipo=01`, `pregunta=P1|P2|P3`, `commit=<sha>`), cada corrida lleva las etiquetas `model_family` (p. ej. `random_forest`, `logreg`, `kmeans`, `dummy`) y `stage_of_work` (`baseline`, `search`, `candidate`), que permiten filtrar y comparar en la interfaz de MLflow; por ejemplo, `tags.stage_of_work = 'candidate'` muestra solo las candidatas. Los parámetros registran también el rango de fechas de cada partición y el hash del manifiesto.

3.2 Tabla comparativa por pregunta

La Tabla 3 consolida, para cada pregunta, el *baseline*, las corridas candidatas y cada corrida hija de la búsqueda anidada, con sus hiperparámetros clave, métricas de validación y prueba, y mejora relativa respecto al *baseline*. La fila sombreada corresponde al modelo elegido.

Tabla 3. *Comparación de cada pregunta contra su baseline (★ = modelo elegido y registrado con @champion).*

Pregunta	Modelo (Run ID)	Hiperparámetros clave	Validación	Prueba	Mejora vs. baseline
P1	Baseline Ridge (b4759af6)	`alpha=1.0`	MAE 0.834	MAE 0.814	Referencia (naive 4 sem.: 0.842)
P1	RF candidato (5e942ce3)	`n_estimators=100`	MAE 0.808	MAE 0.793	−2.6 %
P1	`p1-candidate-rf-n50` (48a18724)	`n_estimators=50`	—	MAE 0.7867; RMSE 1.4420; R^2 0.6672	−3.4 %
P1	`p1-candidate-rf-n100` (f65ed32c)	`n_estimators=100`	—	MAE 0.7862; RMSE 1.4408; R^2 0.6678	−3.4 %
P1 ★	`p1-candidate-rf-n200` (41e42807)	`n_estimators=200`	—	MAE 0.7861; RMSE 1.4402; R^2 0.6681	−3.4 %
P2	Baseline Dummy (ed1f8776)	`strategy=most_frequent`	—	ROC-AUC 0.500; F1-macro 0.494	Referencia
P2	`p2-child-logreg-C0.1` (3338b1bc)	`C=0.1`	F1-macro 0.526388; ROC-AUC 0.8293	Solo la elegida pasa a prueba	Selección en validación
P2	`p2-child-logreg-C10.0` (ffe7a2d8)	`C=10.0`	F1-macro 0.526387; ROC-AUC 0.8293	Solo la elegida pasa a prueba	Selección en validación
P2 ★	`p2-child-logreg-C1.0` (774a24cc); modelo en el padre `P2-parent-search-logreg` (3ad75953)	`C=1.0`, `class_weight=balanced`	F1-macro 0.526389; ROC-AUC 0.8293	ROC-AUC 0.824; F1-macro 0.513; AP 0.128	+64.8 % ROC-AUC
P3	Baseline K-Means random (93caace1)	`init=random`, K=5, `n_init=1`	—	Silueta 0.1591	Referencia
P3 ★	`p3-candidate-k3` (9fa2a199)	`k-means++`, K=3, `n_init=10`	—	Silueta 0.2562	+61.0 %
P3	`p3-candidate-k4` (724f4af5)	`k-means++`, K=4, `n_init=10`	—	Silueta 0.1773	+11.4 %
P3	`p3-candidate-k5` (0a5293df)	`k-means++`, K=5, `n_init=10`	—	Silueta 0.1461	−8.2 %
P3	`p3-candidate-k6` (827a58e2)	`k-means++`, K=6, `n_init=10`	—	Silueta 0.1431	−10.1 %

Los valores de las corridas hijas provienen de las búsquedas anidadas lanzadas con `mlflow run . -e search` y se muestran en las Figuras 1 a 3. Las métricas de las hijas de P1 y P3 se calculan sobre el periodo de prueba y las de las hijas de P2, sobre validación.

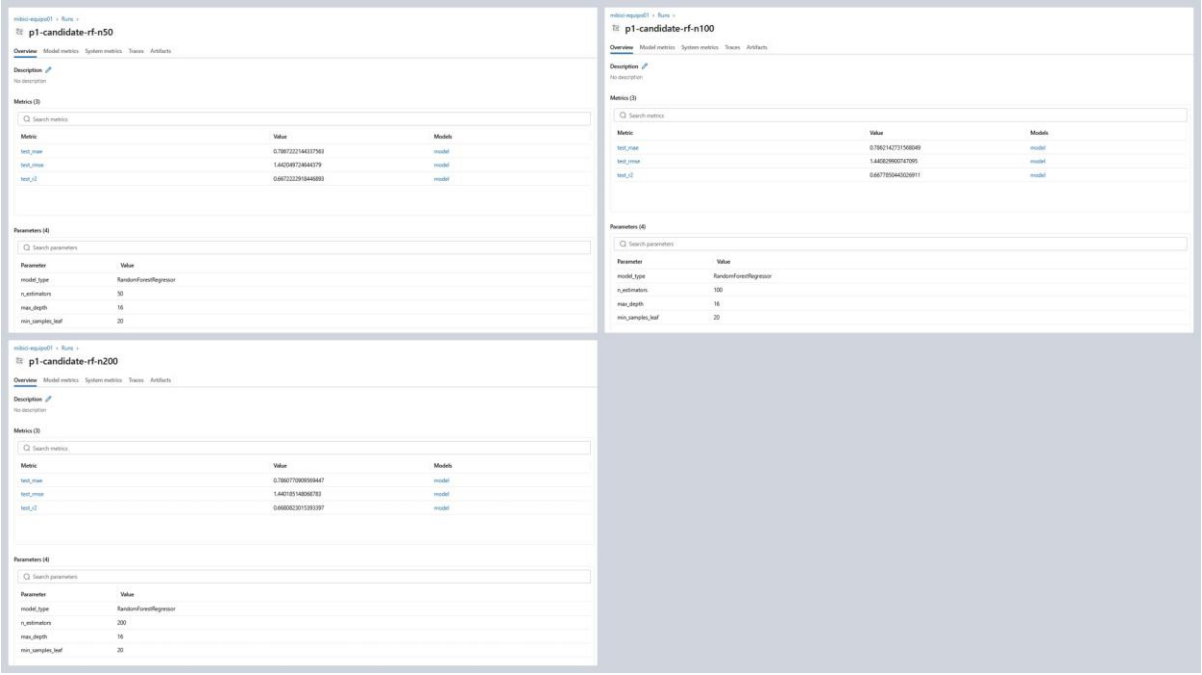


Figura 1. Corridas hijas de `p1-parent-search-rf`: `test_mae`, `test_rmse` y `test_r2` para `n_estimators` = 50, 100 y 200.

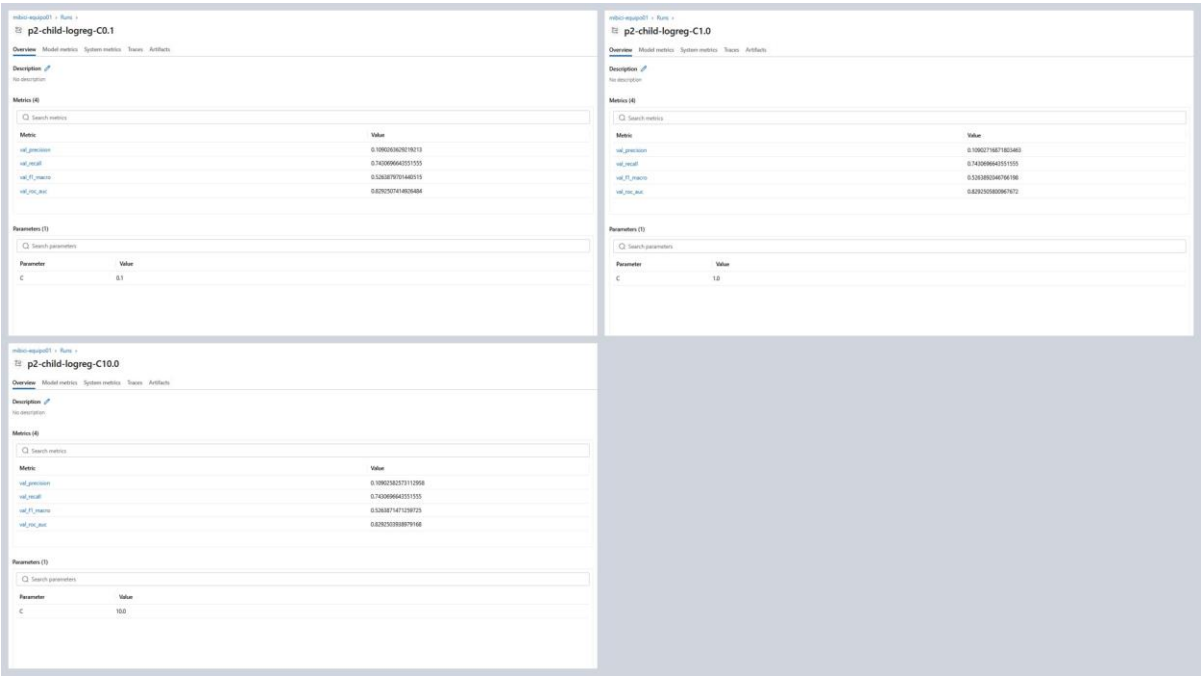


Figura 2. Corridas hijas de `P2-parent-search-logreg`: métricas de validación para C = 0.1, 1.0 y 10.0.

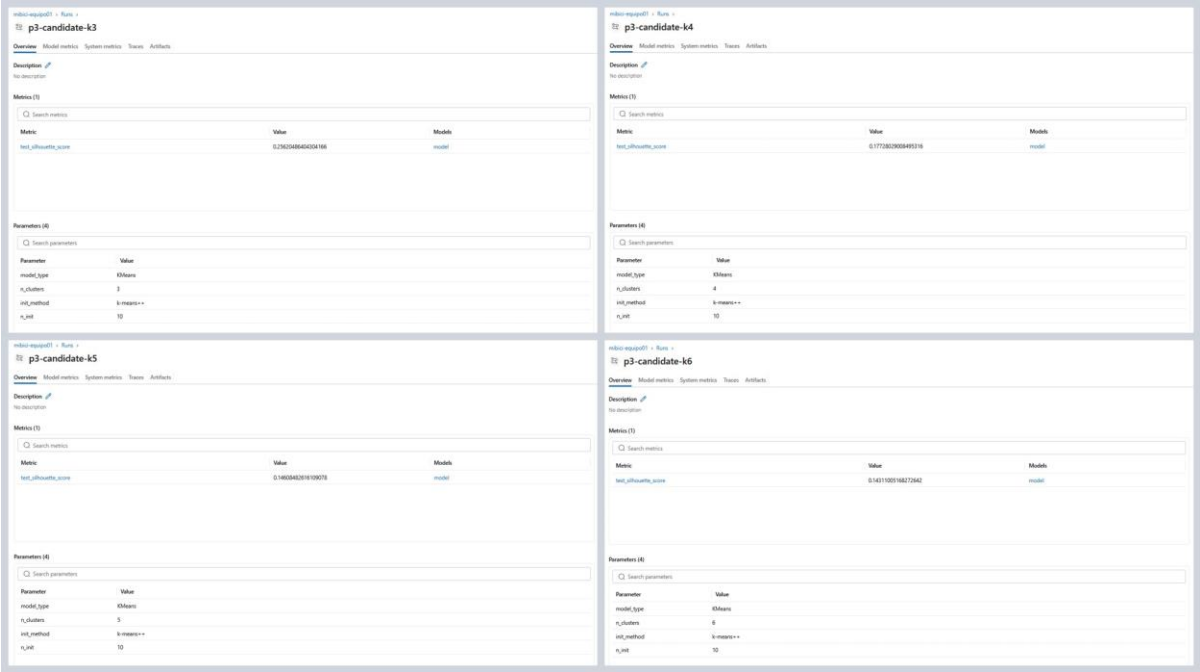


Figura 3. Corridas hijas de `p3-parent-search-kmeans`: `test_silhouette_score` para K = 3, 4, 5 y 6.

3.3 Análisis de la comparación

P1. El Random Forest mejora de forma consistente al Ridge en validación (0.808 vs. 0.834) y en prueba (0.793 vs. 0.814), lo que confirma que la mejora no es producto de la partición. La búsqueda anidada muestra una curva prácticamente plana a partir de 50 árboles (MAE de 0.7867, 0.7862 y 0.7861; diferencias menores a 0.001), es decir, el modelo ya está saturado en capacidad y aumentar árboles solo eleva el costo de entrenamiento. La ganancia frente al baseline proviene de las interacciones no lineales entre hora, día y estación que el modelo lineal no captura.

P2. El clasificador trivial obtiene ROC-AUC de 0.500, como se espera de un modelo que siempre predice la clase mayoritaria en un problema desbalanceado (2.6 % de horas-estación en riesgo en prueba). La regresión logística con pesos balanceados eleva el ROC-AUC a 0.824, por lo que sí ordena correctamente las horas de mayor riesgo. Las tres hijas prácticamente empatan en validación (F1-macro 0.526388, 0.526389 y 0.526387; ROC-AUC 0.8293), lo que indica que la regularización no es el factor limitante; el F1-macro moderado (0.513) refleja la dificultad de la clase minoritaria y deja margen para modelos no lineales en etapas posteriores.

P3. Con la misma semilla, pasar de inicialización aleatoria a K-Means++ con 10 reinicios estabiliza los centroides, y la búsqueda sobre K muestra que la cohesión cae monótonamente al aumentar K (0.256 → 0.177 → 0.146 → 0.143). K=3 es, por lo tanto, el número de grupos mejor separados según el coeficiente de silueta.

3.4 Registro de artefactos

Las corridas candidatas guardan los artefactos que respaldan su resultado, organizados en subcarpetas del *artifact store*. La Tabla 4 detalla qué registra cada pregunta:

Tabla 4. *Artefactos registrados por pregunta.*

Artefacto	Subcarpeta	Implementación

Artefacto	Subcarpeta	Implementación
Modelo con firma y ejemplo de entrada	`model/`, `model_signature/`	`mlflow.sklearn.log_model` con `infer_signature` e `input_example` (5 filas); la firma y el ejemplo se guardan también como JSON.
Versión de los datos	`data/`, pestaña *Datasets* y parámetros	P1: `data/manifest.json` (hashes SHA-256) como artefacto y conjunto de entrenamiento con `mlflow.log_input`. P2: `mlflow.log_input` y hashes del manifiesto y del Parquet como parámetros (`data_manifest_sha256`, `data_parquet_sha256`). P3: hashes como parámetros de la corrida padre de la búsqueda.
Gráficas de desempeño	`plots/`, `classification_artifacts/`, `clustering_artifacts/`	P1: real vs. predicho y residuales. P2: matriz de confusión y curva precisión-recall. P3: mapa geográfico de estaciones y distribución de clústeres.
Configuración de la corrida	`config/` y parámetros	P1: archivo de configuración en `config/`. P1, P2 y P3: partición temporal, semilla e hiperparámetros registrados como parámetros de la corrida, leídos de `config.py`.

Ningún artefacto contiene datos personales de los usuarios de MiBici: las figuras y tablas se construyen a nivel estación-hora o estación, sin identificadores de usuario, y las credenciales viven solo en `.env`, que no se versiona.

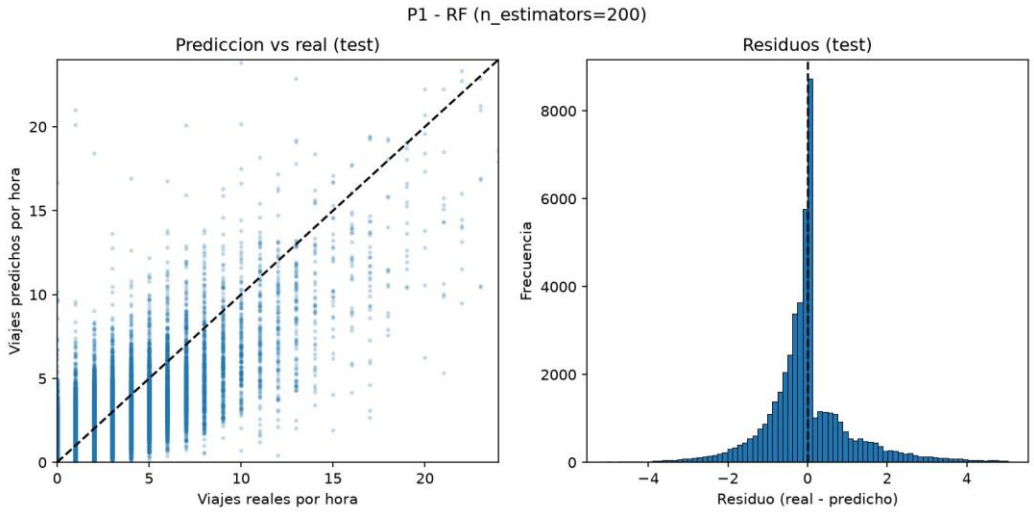


Figura 4. Artefacto de P1: valores reales contra predichos y distribución de residuales del Random Forest (n=200).

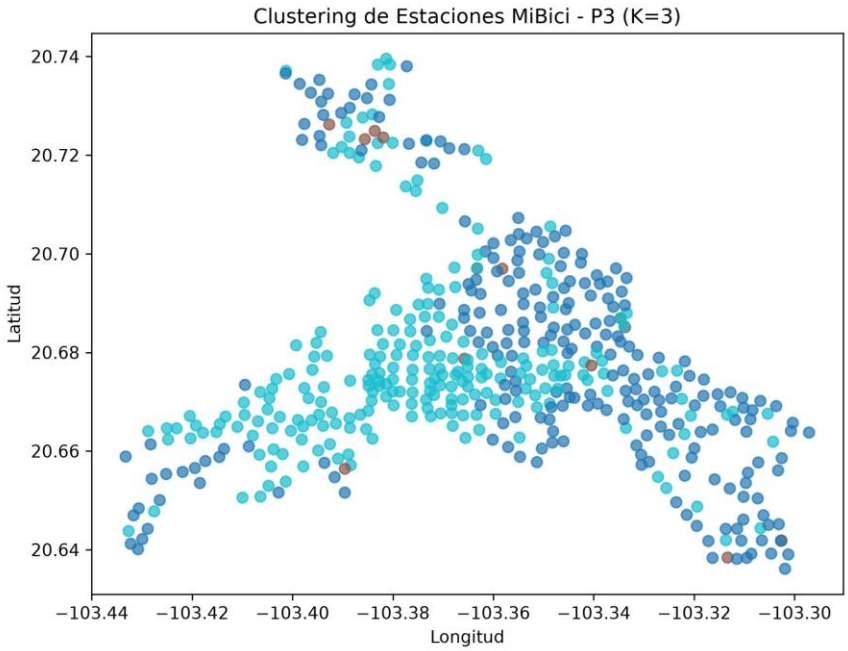


Figura 5. Artefacto de P3: mapa de estaciones de MiBici coloreadas según su grupo (K-Means++, K=3).

4. Justificación de la selección del mejor modelo

La elección se basó en la mejora sobre el *baseline*, en la métrica ligada a la decisión de negocio de cada pregunta, en la estabilidad de los resultados y en el costo de entrenamiento e inferencia.

4.1 P1 — Demanda: Random Forest (n_estimators=200)

- **Desempeño:** menor `test_mae` (0.7861), 3.4 % menos error que el Ridge (0.814) y 6.6 % menos que la referencia ingenua de 4 semanas (0.842).
- **Negocio:** el MAE se interpreta directamente como viajes de error por estación-hora; mantenerlo por debajo de un viaje es suficiente para planear el rebalanceo diario.
- **Estabilidad y costo:** los tres candidatos quedaron prácticamente empatados (0.7867 / 0.7862 / 0.7861), lo que muestra un modelo estable ante el número de árboles. Se eligió n=200 por menor MAE; su costo de inferencia es despreciable para una planeación diaria por lotes, aunque n=50 sería una alternativa válida si el costo de entrenamiento se volviera restrictivo.

4.2 P2 — Riesgo: Regresión logística (C=1.0, class_weight=balanced)

- **Desempeño:** ROC-AUC de prueba de 0.824 frente a 0.500 del baseline (+64.8 %).
- **Negocio:** el costo de no anticipar una estación vacía o llena (falso negativo) es mayor que el de una visita de rebalanceo innecesaria; los pesos balanceados priorizan la clase en riesgo. Además, sus coeficientes son interpretables para los operadores logísticos y la inferencia es instantánea.
- **Estabilidad y costo:** las tres hijas obtuvieron prácticamente el mismo F1-macro de validación (0.526388, 0.526389 y 0.526387), lo que indica que el desempeño es insensible a la regularización en este rango. La búsqueda eligió `C=1.0` (`best_C`, Figura 12), cuyo F1-macro es el más alto por una diferencia en la sexta cifra decimal y que corresponde al valor por defecto de scikit-learn, sin regularización adicional; el modelo se evaluó una sola vez en prueba. La búsqueda completa tarda unos 5 minutos.

4.3 P3 — Grupos: K-Means++ (K=3, n_init=10)

- **Desempeño:** silueta de 0.2562 frente a 0.1591 del baseline con inicialización aleatoria (+61.0 %), y el valor más alto de la búsqueda sobre K.
- **Negocio:** tres perfiles horarios de uso (zonas residenciales, universitarias y de uso mixto) son fáciles de comunicar y de convertir en rutas de rebalanceo diferenciadas.
- **Estabilidad y costo:** K-Means++ con 10 reinicios y semilla fija produce agrupamientos repetibles entre ejecuciones; el entrenamiento toma menos de un minuto. Como limitación, una silueta de 0.256 indica grupos con traslape moderado, algo esperable en patrones de movilidad urbana.

5. Parte C. Model Registry y ciclo de vida

5.1 Modelos registrados

El punto de entrada `register` (`mibici/modeling/register.py`) busca el mejor run de cada pregunta según su métrica de prueba, priorizando las corridas con `stage_of_work=candidate`, lo registra con un nombre consistente y escribe en la descripción del modelo y de la versión su linaje completo: experimento, run de origen, *commit* y hashes del manifiesto y del Parquet de datos. Es idempotente: no registra dos veces el mismo run.

Tabla 5. *Modelos en el Model Registry con su versión en producción.*

Modelo registrado	Criterio de selección	Alias	Descripción / linaje
`mibici-equipo01-p1-demanda`	`test_mae` (menor)	`@champion`	Versión 2. Run `p1-candidate-rf-n200` (41e42807…), commit 047a918, test_mae=0.7861.
`mibici-equipo01-p2-riesgo`	`test_roc_auc` (mayor)	`@champion`	Versión 2. Run `P2-parent-search-logreg` (3ad75953…), commit 483c6f4, manifest ceff081f… / parquet fbb830d0…, test_roc_auc=0.8240.
`mibici-equipo01-p3-grupos`	`test_silhouette_score` (mayor)	`@champion`	Versión 2. Run `p3-candidate-k3` (9fa2a199…), commit 047a918, test_silhouette_score=0.2562.

5.2 Esquema de ciclo de vida: aliases

Se adoptó el esquema vigente de **aliases** de MLflow en lugar de las etapas clásicas (*Staging*/*Production*), que MLflow marca como obsoletas desde la versión 2.9. La versión en producción de cada modelo se identifica con el alias `@champion`, asignado por `register.py` con `set_registered_model_alias`. Las razones fueron:

- **Desacoplamiento:** los consumidores (la API de servido y el monitoreo de las siguientes etapas) cargan siempre `models:/<nombre>@champion` sin conocer el número de versión; promover un modelo nuevo solo mueve el alias.
- **Rollback inmediato:** volver a la versión anterior consiste en reasignar el alias, sin redesplegar código.
- **Preparación para A/B testing:** el esquema admite un segundo alias `@challenger` para la siguiente mejor versión, que se usará al comparar modelos del registro en el módulo 3.

La versión en producción se verifica cargándola por nombre y alias:

```
import mlflow
mlflow.set_tracking_uri("http://127.0.0.1:5000")
modelo = mlflow.pyfunc.load_model("models:/mibici-equipo01-p2-riesgo@champion")
print(modelo.metadata.run_id)   # 3ad7595371f64f3083e42473543aafd2
```

6. Evidencia en la interfaz de MLflow

Las capturas de esta sección corresponden al paso 10 de la actividad y se versionan en `reports/figures/mlflow_evidencia/`.

6.1 Comparación de corridas con nested runs

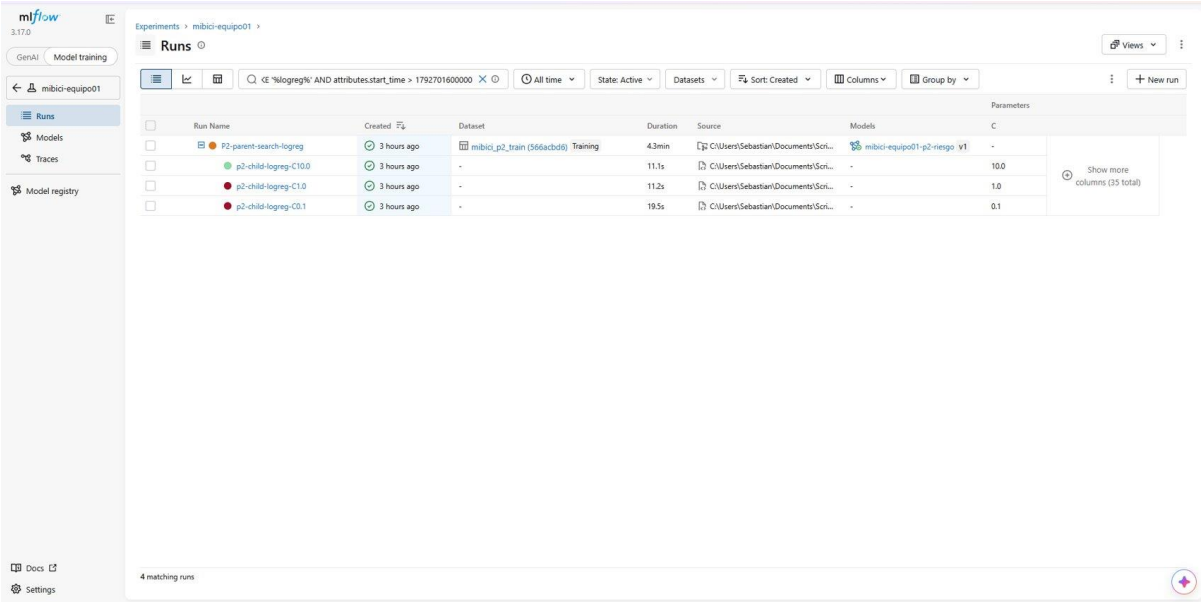


Figura 6. Búsqueda anidada de P2: corrida padre `P2-parent-search-logreg` con sus tres corridas hijas (C = 0.1, 1.0 y 10.0), dataset registrado y modelo vinculado a `mibici-equipo01-p2-riesgo` v1.

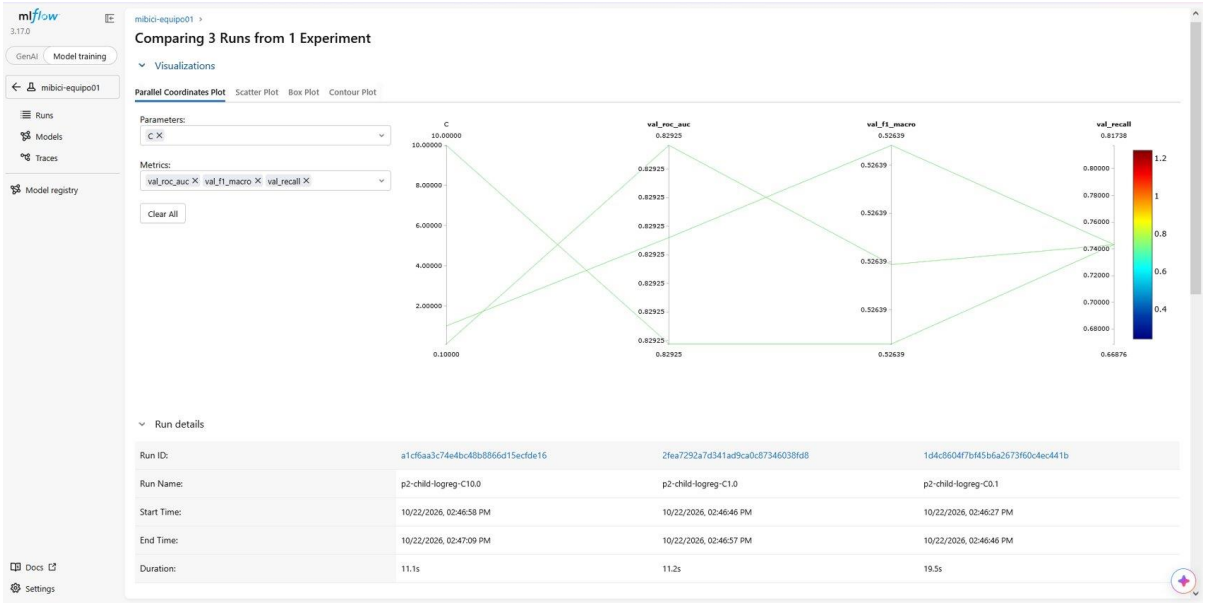


Figura 7. Vista Compare de las corridas hijas de P2: gráfica de coordenadas paralelas entre C y las métricas de validación.

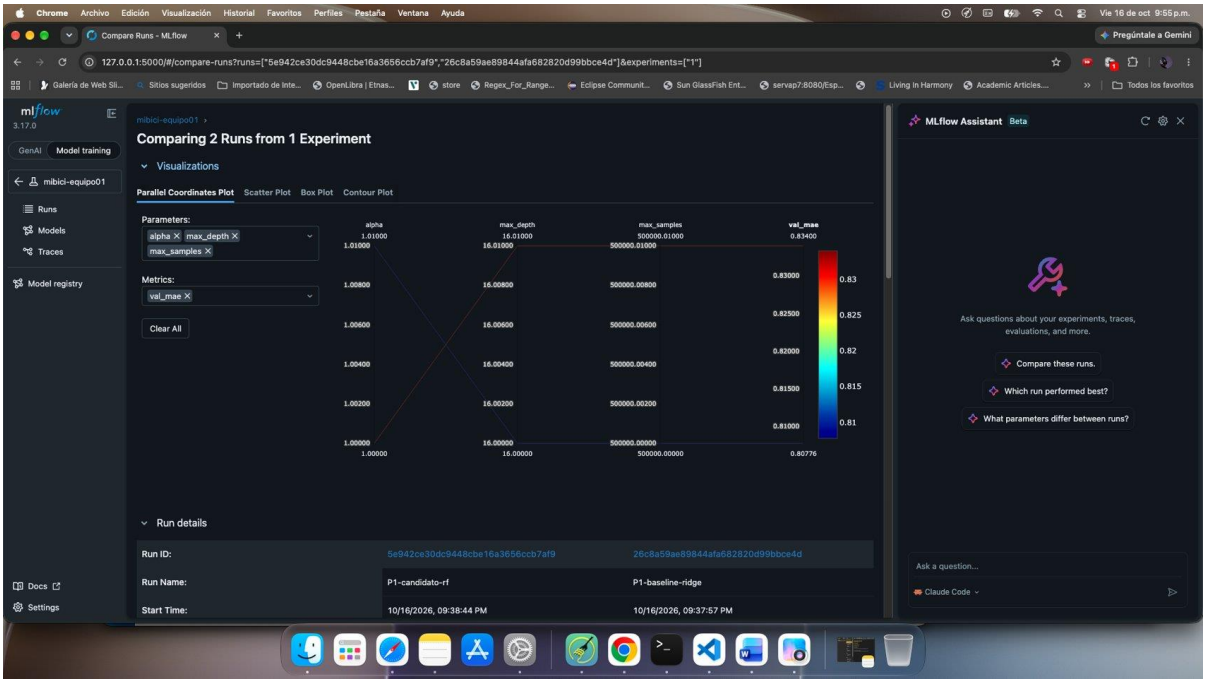


Figura 8. Comparación de corridas de P1 (baseline Ridge contra candidata Random Forest) en la interfaz de MLflow.

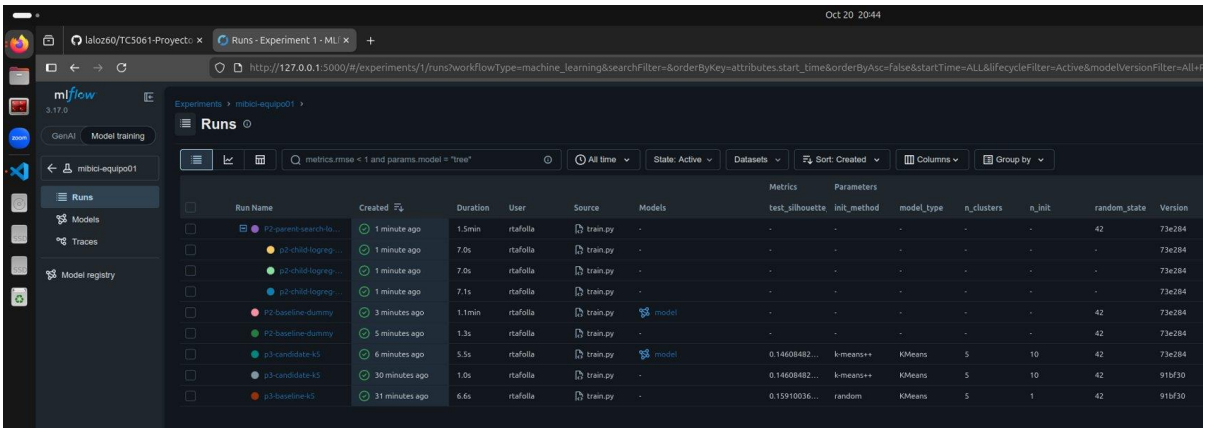


Figura 9. Vista del experimento `mibici-equipo01` con corridas anidadas y filtradas por pregunta, con parámetros y métricas de P3 visibles.

6.2 Detalle de artefactos de una corrida candidata

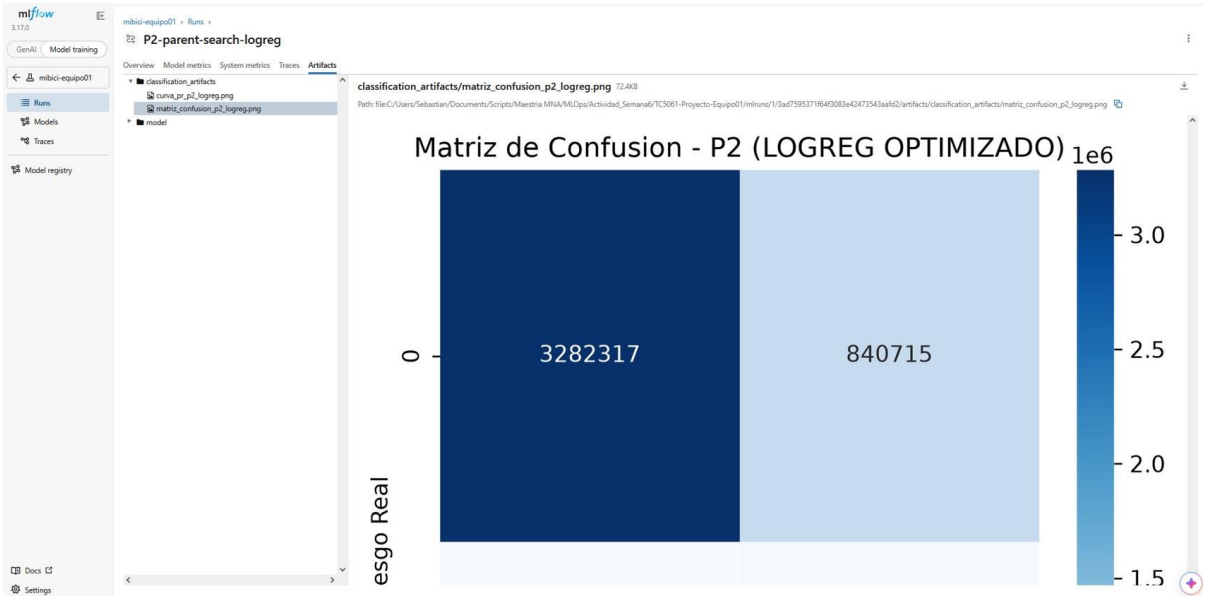


Figura 10. Artefactos de la corrida candidata de P2: modelo con firma, `classification_artifacts/` con la matriz de confusión y manifiesto de datos.

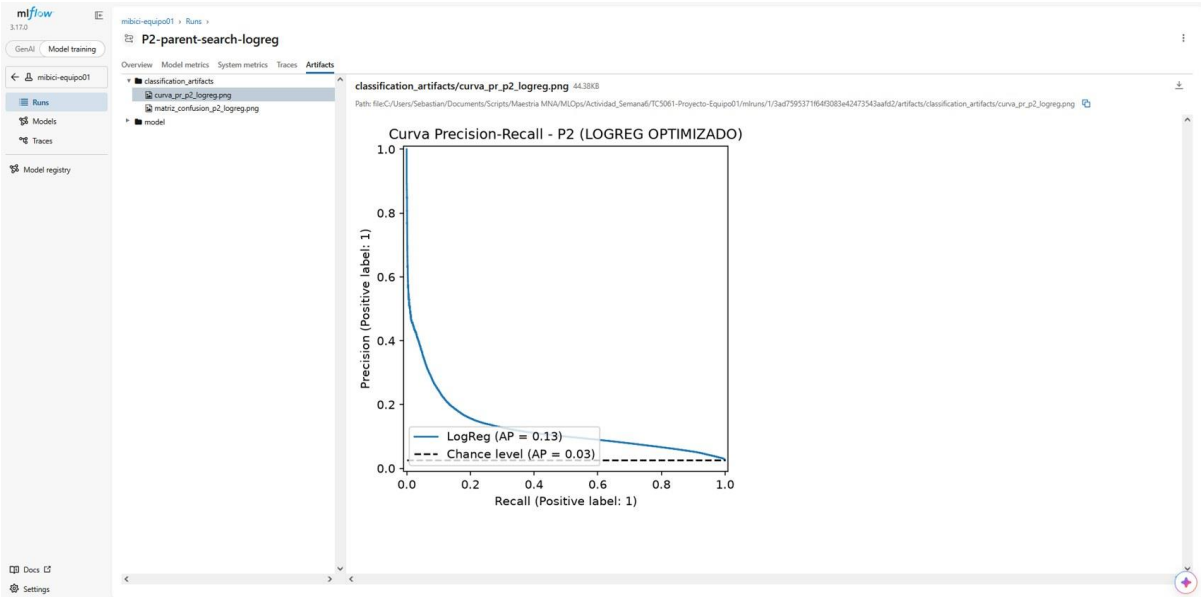


Figura 11. Curva precisión-recall de P2 registrada en `classification_artifacts/` de `P2-parent-search-logreg` (AP = 0.13 frente a 0.03 del nivel de azar).

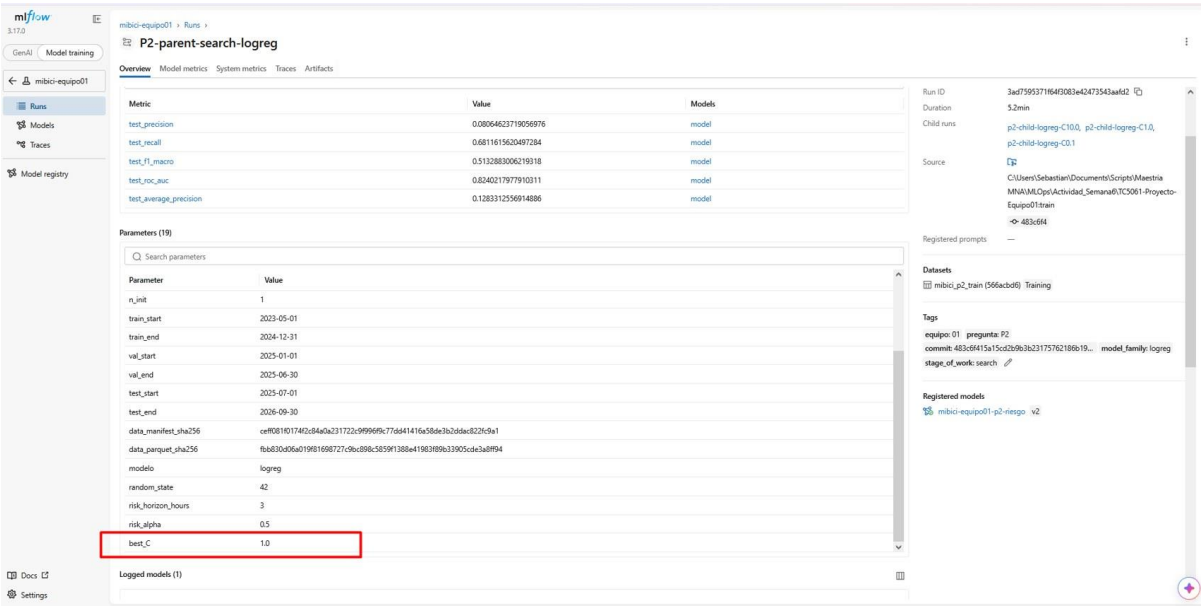


Figura 12. Resumen de `P2-parent-search-logreg`: métricas de prueba (incluida `test_average_precision`), rangos de la partición temporal, hashes del manifiesto y del Parquet, `best_C`, corridas hijas, dataset, etiquetas y modelo registrado (`mibici-equipo01-p2-riesgo` v2).

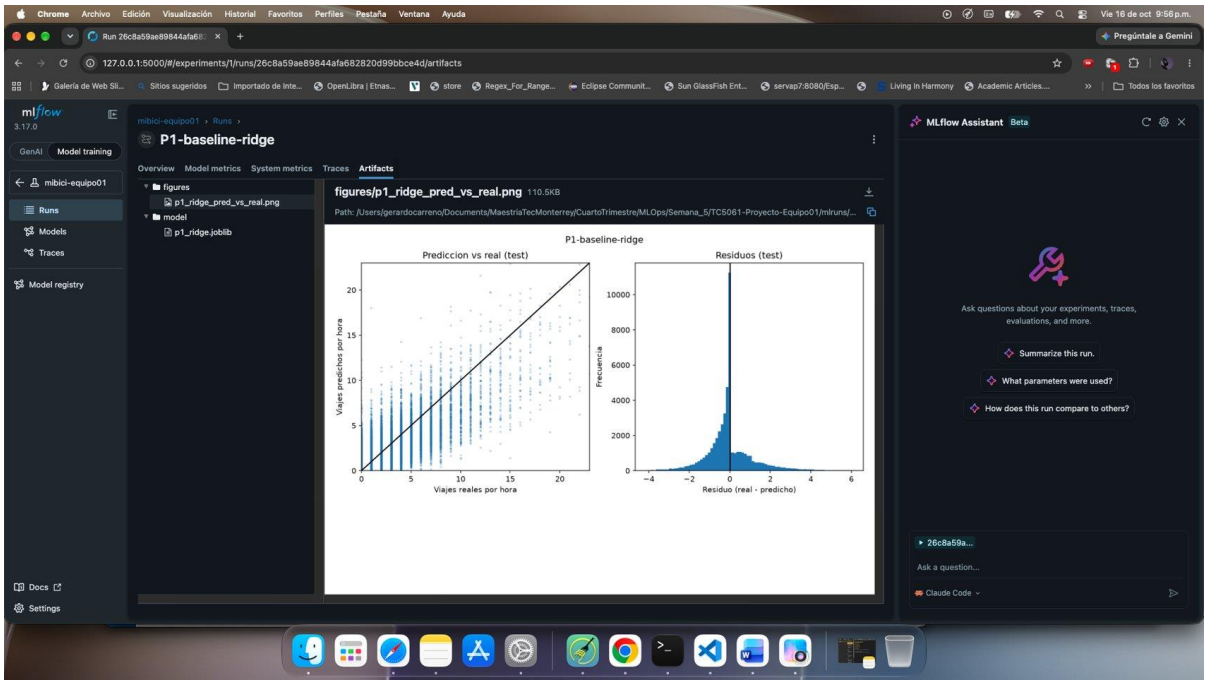


Figura 13. Artefactos de P1: gráfica de real contra predicho registrada en la subcarpeta de gráficas.

6.3 Modelos registrados en producción con su linaje

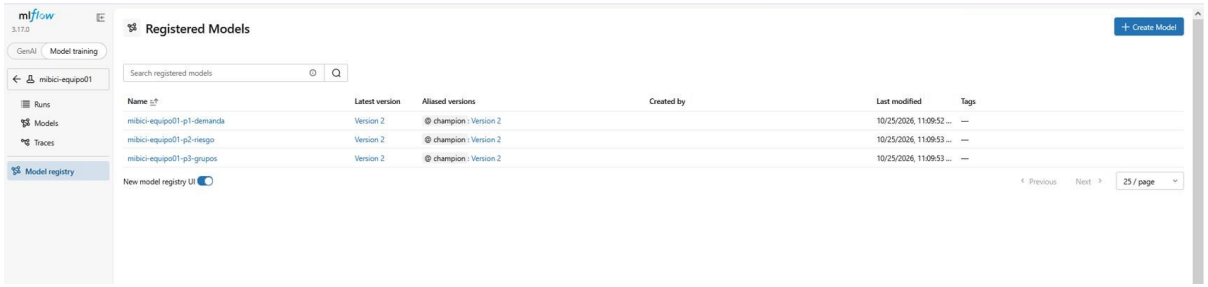


Figura 14. Model Registry con los tres modelos (`p1-demanda`, `p2-riesgo`, `p3-grupos`), cada uno con su versión 2 bajo el alias `@champion`.

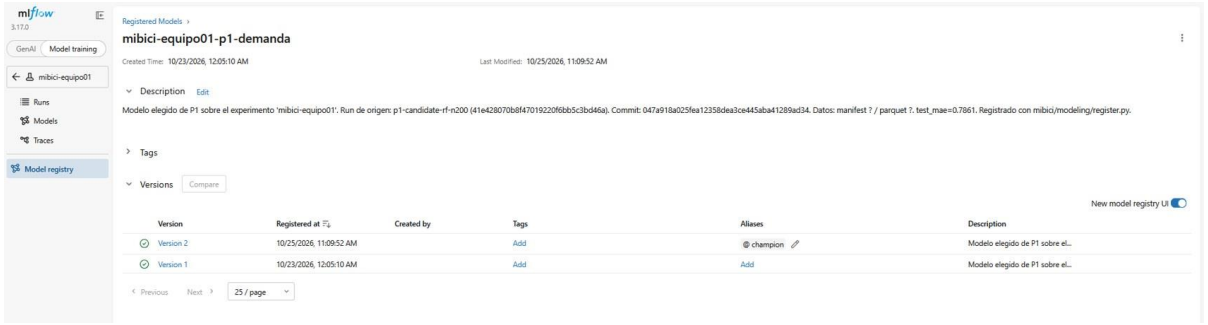


Figura 15. Detalle de `mibici-equipo01-p1-demanda`: la versión 2, con alias `@champion`, proviene de `p1-candidate-rf-n200`; la descripción documenta run de origen, commit y métrica.

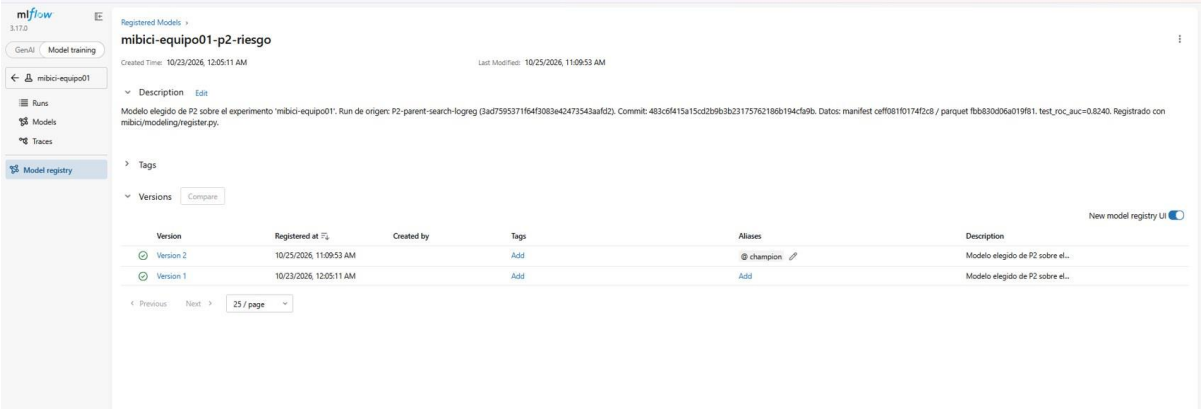


Figura 16. Detalle de `mibici-equipo01-p2-riesgo`: la versión 2, con alias `@champion`, documenta run de origen, commit y hashes del manifiesto y del Parquet.

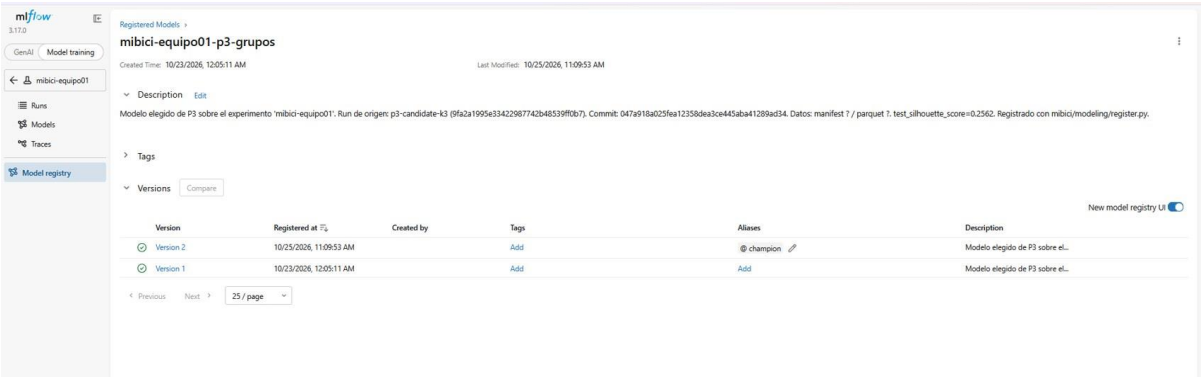


Figura 17. Detalle de `mibici-equipo01-p3-grupos`: la versión 2, con alias `@champion`, proviene de `p3-candidate-k3`; la descripción documenta run de origen, commit y métrica.

7. Flujo de trabajo con Git y GitHub

Todo el trabajo entró a la rama `main`, protegida con *Require pull request before merging* y al menos una aprobación de otro integrante. Las ramas siguen el formato `feature/<nombre>`, `fix/<issue>` o `docs/<tema>`, y los mensajes usan *Conventional Commits* (`feat:`, `fix:`, `docs:`, `chore:`, `exp:`). Cada PR se vinculó a un issue de la entrega y se verificó con `ruff` y `pytest` antes de fusionarse. El ciclo de revisión acordado fue Gerardo → Gemma → Jesús → Sebastián → Gerardo.

Tabla 6. *Pull Requests fusionados en esta entrega.*

PR	Rama	Autor de los cambios	Aporte
#75	`docs/servidor-tracking-mlflow`	Gerardo Carreño	Servidor de tracking con SQLite y `MLFLOW_TRACKING_URI`
#76	`feat/mlproject-entregable2`	Gerardo Carreño	`MLproject`, `python_env.yaml` y `mlflow_entry.py`
#77, #78	`feature/p3-clustering`, `feature/issue-43-recorrer-p3`	Gemma Martínez, Jesús Tafolla	P3 con nested runs, firma y artefactos
#79	`feature/issue-64-p2-nested-runs`	Jesús Tafolla	Búsqueda anidada de P2 y corrección de fuga de prueba

PR	Rama	Autor de los cambios	Aporte
#80, #82	`feat/registry-modelos`, `fix/registry-fallback`	Gerardo Carreño	Registro de modelos con linaje
#81, #90	`feature/p1-demanda-nested-runs`	Gemma Martínez	Búsqueda anidada de P1 y subcarpetas de artefactos
#83, #84, #88	`chore/limpieza-ruff`, `chore/evidencias-…`, `fix/issue-86-…`	Gerardo Carreño	Limpieza de lint, figuras oficiales, aviso de ingesta
#85	`docs/issue-62-readme-mlflow-run`	Sebastián Mucito	README con comandos `mlflow run` y validación en clon limpio
#87	`fix/issue-1-champion-alias`	Jesús Tafolla	Asignación del alias `@champion` en `register.py`
#89	`docs/issue-73-capturas-mlflow`	Sebastián Mucito	Capturas de MLflow del paso 10
#91	`docs/readme-alias-champion`	Sebastián Mucito	README: alias `@champion` y responsabilidades por integrante
#92	`feat/p2-curva-precision-recall`	Sebastián Mucito	Curva precisión-recall y *average precision* de P2; capturas de linaje de P1 y P3
#93	`feat/mlproject-search`	Sebastián Mucito	Entry point `search` para lanzar las búsquedas anidadas con `mlflow run`; capturas con los modelos en versión 2

Los cuatro integrantes tienen *commits* en esta entrega y cada uno abrió al menos un PR; las aprobaciones de cada PR pueden consultarse en la pestaña *Pull requests* del repositorio.

8. Conclusiones

El repositorio pasó de un conjunto de scripts a un MLflow Project que cualquier integrante reproduce desde un clon limpio con `mlflow run`, lo que quedó comprobado en una validación documentada paso a paso. La experimentación organizada con *nested runs* y etiquetas hizo que la comparación contra el *baseline* fuera sistemática y auditable: los tres modelos elegidos superan su línea base (−3.4 % de MAE en P1, +64.8 % de ROC-AUC en P2 y +61.0 % de silueta en P3). Finalmente, cada modelo en producción está en el Model Registry con su linaje hasta el *commit* y la versión de datos, y se consume por alias, lo que deja lista la base para las etapas de servido, monitoreo y entrenamiento continuo del proyecto.

9. Declaración de uso de inteligencia artificial

En el desarrollo de esta entrega el equipo utilizó Claude Code (Anthropic, 2026), un asistente de programación basado en inteligencia artificial, como apoyo para la corrección de código. Su uso se limitó a identificar y proponer correcciones de errores, depurar fallas de ejecución y sugerir mejoras de limpieza y estilo en los módulos del proyecto.

Todas las propuestas generadas por la herramienta fueron revisadas, probadas y, en su caso, ajustadas por los integrantes del equipo antes de integrarse al repositorio. Cada cambio siguió el

mismo flujo que el resto del trabajo: se incorporó mediante un *pull request*, se verificó con `ruff` y `pytest`, y fue aprobado por al menos un integrante distinto de su autor. El diseño experimental, la selección de los modelos, la interpretación de los resultados y las decisiones de negocio descritas en este reporte son responsabilidad del equipo.

Referencias

Anthropic. (2026). *Claude Code* [Asistente de programación basado en inteligencia artificial]. https://claude.com/product/claude-code

Huyen, C. (2022). *Designing machine learning systems: An iterative process for production-ready applications*. O'Reilly Media.

Kumar, N. (2021). *Machine learning engineering with MLflow*. Packt Publishing.

MiBici. (s.f.). *Datos abiertos*. Sistema de Bicicleta Pública del Área Metropolitana de Guadalajara. https://mibici.net/es/datos-abiertos

MLflow. (s.f.-a). *MLflow Model Registry*. MLflow Documentation. https://mlflow.org/docs/latest/ml/model-registry/

MLflow. (s.f.-b). *MLflow Projects*. MLflow Documentation. https://mlflow.org/docs/latest/ml/projects/

MLflow. (s.f.-c). *MLflow Tracking*. MLflow Documentation. https://mlflow.org/docs/latest/ml/tracking/

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., & Dennison, D. (2015). Hidden technical debt in machine learning systems. En *Advances in Neural Information Processing Systems 28* (pp. 2503–2511). Curran Associates.

Treveil, M., Omont, N., Stenac, C., Lefevre, K., Phan, D., Zentici, J., Lavoillotte, A., Miyazaki, M., & Heidmann, L. (2020). *Introducing MLOps: How to scale machine learning in the enterprise*. O'Reilly Media.

Wilson, B. (2022). *Machine learning engineering in action*. Manning Publications.